Simon Edmunds
12590227

CS 5001: Natural Language Processing Spring 2026

Assignment 2

Due 11:59 PM April 21, 2026

Questions:

1. Consider the dataset below, which has 10 training examples and 2 features:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad Y = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Now, use Naïve Bayes model to classify the following 2 test examples:

$$X_{test} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

a) Compute the estimates of the class prior probabilities:

P(Y = 1) = 6/10 = 0.6

P(Y = 0) = 4/10 = 0.4

b) Compute the estimates of the four conditional probabilities required by Naïve Bayes for this example

(Probabilities were found by counting values in the dataset that meet the condition)

P(X1 = 1 | Y = 1) = 3/6 = 0.5

P(X2 = 1 | Y = 1) = 4/6 = 0.667

P(X1 = 1 | Y = 0) = 4/4 = 1.0

P(X2 = 1 | Y = 0) = 1/4 = 0.25

2.

a) Compute the vector space similarity between the query "digital cameras" and the document "digital cameras and video cameras" by filling out the empty columns in the table below. Assume $N = 10^7$, logarithmic term weighting (wf columns) for query and document, idf weighting for the query only and cosine normalization for the document only. Treat 'and' as a stop word. Enter term counts in the tf columns. What is the final similarity score between the query and the document?

Word	Query					Document			qi*di
	tf	wf	df	idf	qi=wf-idf	tf	wf	di=normalized wf	
digital	**1**	**1**	10,000	**3**	**3**	**1**	**1**	**0.520**	**1.560**
video	**0**	**0**	100,000	**2**	**0**	**1**	**1**	**0.520**	**0**
Cameras	**1**	**1**	50,000	**2.301**	**2.301**	**2**	**1.301**	**0.677**	**1.558**

Note: I didn't show work in the table, but to find di, I divided wf by the square root of the sum of squared wf values

The similarity is the sum of the dot product at the end, or 1.560 + 0 + 1.558 = **3.118**

b) Consider the following tables showing the idf values for terms car, auto, insurance and best in the Reuters collection; as well as the term frequencies for 3 documents denoted Doc1, Doc2, Doc3

Table 1: tf values

	Doc1	Doc2	Doc3
car	27	4	24
insurance	0	33	29
best	14	0	17
auto	3	33	0

Table 2: idf values

	idft
car	1.65
insurance	1.62
best	1.5
auto	2.08

(i) Compute the tf-idf weights for the terms car, insurance, best, auto for each document, using the idf values from Table 2.

Doc1:

- car: 27 * 1.65 = **44.55**
- insurance: 0 * 1.62 = **0**
- best: 14 * 1.5 = **21**
- auto: 3 * 2.08 = **6.24**

Doc2:

- car: 4 * 1.65 = **6.6**
- insurance: 33 * 1.62 = **53.46**
- best: 0 * 1.5 = **0**
- auto: 33 * 2.08 = **68.64**

Doc3:

- car: 24 * 1.65 = **39.6**
- insurance: 29 * 1.62 = **46.98**
- best: 17 * 1.5 = **25.5**
- auto: 0 * 2.08 = **0**

(ii) Using the tf-idf weights computed in the previous part, compute the Euclidean normalized document vectors for each of the documents, where each vector has four components, one for each of the four terms. If the vector is [x1, x2, x3, …., xn], the Euclidean length of this vector is $l = \sqrt{\sum_{i=1}^{n} x_i^2}$ where *n* is the number of elements in the vector. So, after Euclidean normalization the vector becomes $[\frac{x_1}{l}, \frac{x_2}{l}, \frac{x_3}{l}, \ldots, \frac{x_n}{l}]$.

Doc1:

$$l = \sqrt{44.55^2 + 0^2 + 21^2 + 6.24^2} = 49.65$$

- car: 44.55 / 49.65 = 0.897
- insurance: 0 / 49.65 = 0
- best: 21 / 49.65 = 0.423
- auto: 6.24 / 49.65 = 0.126

Normalized Vector: [0.897, 0, 0.423, 0.126]

Doc2:

$$l = \sqrt{6.6^2 + 53.46^2 + 0^2 + 68.64^2} = 87.25$$

- car: 6.6 / 87.25 = 0.076
- insurance: 53.46 / 87.25 = 0.613
- best: 0 / 87.25 = 0
- auto: 68.64 / 87.25 = 0.787

Normalized Vector: [0.076, 0.613, 0, 0.787]

Doc3:

$$l = \sqrt{39.6^2 + 46.98^2 + 25.5^2 + 0^2} = 66.52$$

- car: 39.6 / 66.52 = **0.595**
- insurance: 46.98 / 66.52 = **0.706**
- best: 25.5 / 66.52 = **0.383**
- auto: 0 / 66.52 = **0**

Normalized Vector: [0.595, 0.706, 0.383, 0]

With document vectors as computed above, rank the three documents with respect to the query "car insurance", for each of the following cases of term weighting in the query:

- Binary term weighting, where the weight of a term is 1 if present in the query, 0 otherwise.

  Query vector: [1, 1, 0, 0]
  - Doc1: 0.897 + 0 = 0.897
  - Doc2: 0.076 + 0.613 = 0.689
  - Doc3: 0.595 + 0.706 = 1.301

  Ranking (Best to worst): Doc3, Doc1, Doc2

- Euclidean normalized idf.
  Idf vector: [1.65, 1.62, 0, 0]
  $l = \sqrt{1.65^2 + 1.62^2 + 0^2 + 0^2} = 2.312$
  Normalized idf vector: [0.714, 0.701, 0, 0]
  - Doc1: 0.714(0.897) + 0.701(0) = **0.640**
  - Doc2: 0.714(0.076) + 0.701(0.613) = **0.484**
  - Doc3: 0.714(0.595) + 0.701(0.706) = **0.920**

  Ranking (Best to worst): Doc3, Doc1, Doc2

(iii) Rank the three documents with respect to the query "best car insurance" under the *nnn.atc* and the *ntc.atc* weighting scheme.

Idf vector: [1.65, 1.62, 1.5, 0]

$l = \sqrt{1.65^2 + 1.62^2 + 1.5^2 + 0^2} = 2.756$

Normalized idf vector: [0.599, 0.588, 0.544, 0]

- nnn.atc (doc: tf, query: normalized tf-idf)
  - Doc1: 0.599(27) + 0.588(0) + 0.544(14) + 0(3) = **23.789**
  - Doc2: 0.599(4) + 0.588(33) + 0.544(0) + 0(33) = **21.800**
  - Doc3: 0.599(24) + 0.588(29) + 0.544(17) + 0(0) = **40.676**

  Ranking (Best to worst): Doc3, Doc1, Doc2

- ntc.atc (doc: normalized tf-idf, query: normalized tf-idf)
  **NOTE: For this part, I'm using document normalized tf-idf vectors from part ii
  - Doc1: 0.897(0.599) + 0(0.588) + 0.423(0.544) + 0.126(0) = **0.767**
  - Doc2: 0.076(0.599) + 0.613(0.588) + 0(0.544) + 0.787(0) = **0.406**
  - Doc3: 0.595(0.599) + 0.706(0.588) + 0.383(0.544) + 0(0) = **0.980**

  Ranking (Best to worst): Doc3, Doc1, Doc2

3. The analogy prediction task is defined as follows. Given a pair of words (*a,b*) and a third word *c*, choose a fourth word *d* (which is not *a*, *b* or *c*) so that the analogy "a is to b as c is to d" holds. In other words, the relationship between *c* and *d* should be as close as possible to that between *a* and *b*.

You are provided with the Mikolov's analogy dataset "mikolov_analogy_dataset.txt" from [Mikolov et al., 2013; https://arxiv.org/pdf/1301.3781.pdf]. In the attached text file, each line represents one analogy question, in the form of four words <a, b, c, d>.

For example:
Berlin Germany Moscow Russia

A question is counted as correctly answered only if the predicted word is the same as the given word. For example, given the first three words "Berlin Germany Moscow", the task is to predict "Russia".

The groups of relations in the analogy dataset are delimited by lines starting with a colon (:) and for this assignment, you should only consider the following 10 groups - capital-common-countries, capital-world, currency, city-in-state, family, gram1-adjective-to adverb, gram2-opposite, gram3-comparative, gram6-nationality-adjective, and gram8-plural.

Now, consider the following word embedding models

- Word2vec [Mikolov et al., 2013]: https://code.google.com/archive/p/word2vec/
- GloVe [Pennington et al., 2014]: http://nlp.stanford.edu/projects/glove/

a) Use pretrained word embeddings and run the analogy test. Implement the analogy prediction method described above and compare the embeddings on the 10 analogy tasks listed above. Report overall accuracy, precision, and recall. Also mention accuracy, precision and recall per relation group.

I ran tests for both Word2Vec and GloVe (see reports below)

Results for Word2Vec

capital-common-countries
  Accuracy : 0.832
  Precision: 0.832
  Recall   : 0.832
  Correct : 421
  Total   : 506
  Skipped : 0

capital-world
  Accuracy : 0.791
  Precision: 0.791
  Recall   : 0.791
  Correct : 3580
  Total   : 4524
  Skipped : 0

currency
  Accuracy : 0.351
  Precision: 0.351
  Recall   : 0.351
  Correct : 304
  Total   : 866
  Skipped : 0

gram8-plural
  Accuracy : 0.899
  Precision: 0.899
  Recall   : 0.899
  Correct : 1197
  Total   : 1332
  Skipped : 0

city-in-state
  Accuracy : 0.709
  Precision: 0.709
  Recall   : 0.709
  Correct : 1749
  Total   : 2467
  Skipped : 0

family
  Accuracy : 0.846
  Precision: 0.846
  Recall   : 0.846
  Correct : 428
  Total   : 506
  Skipped : 0

gram1-adjective-to-adverb
  Accuracy : 0.285
  Precision: 0.285
  Recall   : 0.285
  Correct : 283
  Total   : 992
  Skipped : 0

gram2-opposite
  Accuracy : 0.427
  Precision: 0.427
  Recall   : 0.427
  Correct : 347
  Total   : 812
  Skipped : 0

gram3-comparative
  Accuracy : 0.908
  Precision: 0.908
  Recall   : 0.908
  Correct : 1210
  Total   : 1332
  Skipped : 0

gram6-nationality-adjective
  Accuracy : 0.899
  Precision: 0.899
  Recall   : 0.899
  Correct : 1438
  Total   : 1599
  Skipped : 0

Overall Word2Vec Results

Accuracy : 0.7335966791644349
Precision: 0.7335966791644349
Recall   : 0.7335966791644349
Correct : 10957
Total   : 14936

Results for GloVe

capital-common-countries
Accuracy : 0.949
Precision: 0.949
Recall : 0.949
Correct : 480
Total : 506
Skipped : 0

capital-world
Accuracy : 0.960
Precision: 0.960
Recall : 0.960
Correct : 4342
Total : 4524
Skipped : 0

currency
Accuracy : 0.158
Precision: 0.158
Recall : 0.158
Correct : 137
Total : 866
Skipped : 0

city-in-state
Accuracy : 0.593
Precision: 0.593
Recall : 0.593
Correct : 1463
Total : 2467
Skipped : 0

family
Accuracy : 0.881
Precision: 0.881
Recall : 0.881
Correct : 446
Total : 506
Skipped : 0

gram1-adjective-to-adverb
Accuracy : 0.226
Precision: 0.226
Recall : 0.226
Correct : 224
Total : 992
Skipped : 0

gram2-opposite
Accuracy : 0.273
Precision: 0.273
Recall : 0.273
Correct : 222
Total : 812
Skipped : 0

gram3-comparative
Accuracy : 0.881
Precision: 0.881
Recall : 0.881
Correct : 1174
Total : 1332
Skipped : 0

gram6-nationality-adjective
Accuracy : 0.926
Precision: 0.926
Recall : 0.926
Correct : 1480
Total : 1599
Skipped : 0

gram8-plural
Accuracy : 0.781
Precision: 0.781
Recall : 0.781
Correct : 1040
Total : 1332
Skipped : 0

Overall GloVe Results

Accuracy : 0.7370112479914301
Precision: 0.7370112479914301
Recall : 0.7370112479914301
Correct : 11008

Total : 14936

b) One notable issue with word embeddings is their tendency to represent antonyms (words with opposite meanings) with similar embeddings. You can verify this by searching for the top 10 most similar words to a few verbs like “increase” or “enter” that have clear antonyms (e.g., “decrease”
and “exit”, respectively) using the cosine similarity. Discuss why embeddings might have this tendency.

Using Word2Vec embeddings, the embeddings for some antonyms are actually closer than some synonyms!

(notebook output):

```
increase->decrease (antonym): 0.8370319
dude->guy (synonym): 0.7975314
```

This happens because embeddings are calculated based on the contexts in which each word appears. Opposites like “increase” and “decrease” usually occur in nearly identical sentences (“prices increase” vs. “prices decrease”), causing the model to assign them very similar representations. This also explains why both models did badly with the “gram2-opposite” category.

c) Design two new types of analogy tests that are not part of Mikolov’s analogy dataset. You need to create your own test questions (3 questions for each type, so in total 6 new questions). Report how well the embeddings perform on your test questions. You are encouraged to be adversarial so that the embeddings might get an accuracy of zero! Discuss any interesting observations you have made in the process.

I made two new analogy tests “Role” and “Levels”, and tested each with both Word2Vec and GloVe (results below):

=== Role Analogy ===
-- Word2Vec (0/3 correct) --
Question: ('buy', 'sell', 'borrow') - Result: borrowing - Correct: lend
Question: ('teacher', 'student', 'employer') - Result: employers - Correct: employee
Question: ('doctor', 'patient', 'lawyer') - Result: Andrew_Levander - Correct: client
-- GloVe (0/3 correct) --
Question: ('buy', 'sell', 'borrow') - Result: borrowing - Correct: lend
Question: ('teacher', 'student', 'employer') - Result: employers - Correct: employee
Question: ('doctor', 'patient', 'lawyer') - Result: lawyers - Correct: client

=== Levels Analogy ===
-- Word2Vec (1/3 correct) --
Question: ('warm', 'hot', 'cool') - Result: Hot - Correct: cold
Question: ('good', 'better', 'bad') - Result: worse - Correct: worse
Question: ('big', 'huge', 'small') - Result: large - Correct: tiny
-- GloVe (1/3 correct) --
Question: ('warm', 'hot', 'cool') - Result: hottest - Correct: cold
Question: ('good', 'better', 'bad') - Result: worse - Correct: worse
Question: ('big', 'huge', 'small') - Result: large - Correct: tiny

Interestingly, both Word2Vec and GloVe failed almost completely for both of my new test analogies. They consistently returned words that were similar/related, but couldn't really understand the relationships I was trying to imply. This shows that the embeddings prefer similarity in words over interpreting the relationship between words.

Share the answers in the form of a Colab notebook.

Here is the sharing link to my Colab notebook (editor privileges)
https://colab.research.google.com/drive/18LJNYDK4ykXsCrIJqH2P5uGWotaeaVoM?usp=sharing